Desenvolupament d'un agent de Deep Reinforcement Learning per a la presa de decisions estratègiques en entorns estocàstics d'informació imperfecta: El cas de Pokémon VGC

Gerard Pascual Fontanilles

March 10, 2026

Contents

1 Marc Teòric i Contextualització **1**
1.1 Contextualització de la IA en Jocs . . . 1
1.1.1 L'Evolució a través de la Informació Perfecta . . . 1
1.1.2 El Salt als Videojocs Moderns . . . 1
1.1.3 Informació Perfecta vs. Informació Imperfecta . . . 2
1.1.4 Processos de Decisió de Markov (MDP i POMDP) . . . 2
1.1.5 Per què Pokémon és un cas d'estudi ideal? . . . 3
1.2 Pokémon: El Joc . . . 4
1.2.1 Orígens i Franquícia . . . 4
1.2.2 Variables de Combat: Estadístiques (Stats) . . . 4
1.2.3 Taula de Tipus, STAB i Categories d'Atacs . . . 6
1.2.4 Velocitat i Ordre d'Execució . . . 8
1.2.5 Condicions d'Estat (Status Conditions) . . . 8
1.2.6 L'Estocasticitat: El Factor d'Incertesa . . . 9
1.3 El Format Competitiu: VGC . . . 9
1.3.1 Orígens i Història . . . 9
1.3.2 En què es diferencia del Pokémon "normal" . . . 9
1.3.3 El Metagame i les Generacions . . . 10
1.4 L'Entorn Virtual: Eines de Simulació . . . 10
1.4.1 Pokémon Showdown . . . 10
1.4.2 Poke-env . . . 11
1.5 Intel·ligència Artificial i Tècniques Computacionals . . . 11
1.5.1 Algorismes de Cerca Clàssics . . . 11
1.5.2 Xarxes Neuronals Artificials . . . 12
1.5.3 Reinforcement Learning (RL) . . . 12
1.5.4 Pokechamp i PokeLLMon: Agents amb Models de Llenguatge . . . 13
1.6 Justificació de la Recerca . . . 13

2 Metodologia: Disseny Experimental **15**
2.1 Entorn de Simulació . . . 15
2.1.1 Arquitectura Interna de Pokémon Showdown . . . 15
2.1.2 Poke-env com a Interfície Python . . . 15
2.1.3 Variables d'Estat Disponibles . . . 16
2.1.4 Infraestructura Paral·lela de Servidors . . . 16
2.2 Complexitat del Problema . . . 17
2.2.1 Diferencies entre Generacions . . . 17
2.2.2 Singles (1 vs 1) vs. Doubles (2 vs 2) . . . 17
2.3 Espai d'Estats i Espai d'Accions . . . 17
2.3.1 Espai d'Estats (S) . . . 17
2.3.2 Espai d'Accions (A) . . . 18
2.4 Mètriques d'Avaluació . . . 18

2.4.1 Mètriques Primàries 18
2.4.2 Mètriques Secundàries 18
2.5 Fases de Desenvolupament i Models de Referència 19
2.5.1 Fase 1: Models de Regles Expertes (Heurístiques) 19
2.5.2 Fase 2: Models d'Aprenentatge Profund 19
2.5.3 Línies de Control (Baselines) 19

3 Implementació d'Agents: De les Fases Heurístiques a l'Aprenentatge Profund 21
3.1 Orquestració Operacional i Entorn Paral·lelitzat 21
3.2 Fase 1.1: Desenvolupament d'Heurístiques per Singles (1vs1) 21
3.3 Fase 1.2: Integració d'Agents Externs (Pokechamp) 22
3.3.1 Agents Heurístics de Pokechamp (1vs1) 22
3.3.2 Agents LLM: Pokechamp i PokeLLMon 22
3.4 Fase 1.2b: L'Ascens al Competitiu Real (Dobles VGC) 23
3.5 Fase 2: Implementació del Pipeline de Reinforcement Learning 24
3.5.1 Vectorització de l'Estat (`StateVectorizer`) 24
3.5.2 Entorn Gymnasium amb Action Masking 24
3.5.3 Reward Shaping (Disseny de la Recompensa) 24
3.5.4 Currículum d'Entrenament en 4 Fases 25
3.5.5 Arquitectura del Model i Infraestructura GPU 25
3.6 Treball Futur: Aprenentatge Basat en Datasets de Partides 25

4 Resultats i Avaluació de les Heurístiques 27
4.1 Infraestructura d'Execució: Gestió de Recursos 27
4.2 Avaluació Exhaustiva al Format 1 vs 1 (Singles) 28
4.3 Avaluació al Format 2 vs 2 (Doubles) 29

Abstract

Els videojocs d'estratègia complexos constitueixen un banc de proves valuós per a la recerca en Intel·ligència Artificial. En contrast amb jocs d'informació perfecta (com els escacs o el Go), els entorns amb informació parcial i dinàmiques estocàstiques presenten reptes addicionals significatius en predicció i planificació. Aquest Treball de Final de Màster proposa el disseny, implementació i avaluació d'un agent basat en Deep Reinforcement Learning (DRL) per al format competitiu VGC de Pokémon, utilitzant el simulador de codi obert Pokémon Showdown [13] i la llibreria poke-env [16] com a entorn d'entrenament.

El problema s'aborda com un Procés de Decisió de Markov Parcialment Observable (POMDP) [20], caracteritzat per un espai d'estats i d'accions d'alta dimensionalitat i múltiples fonts d'incertesa (informació oculta sobre l'equip rival i resultats estocàstics dels moviments). L'objectiu no és la resolució perfecta del joc, sinó l'aproximació d'una política d'actuació que maximitzi l'esperança de victòria sota incertesa.

La proposta integra: (1) una representació vectorial de l'estat que inclou estadístiques, tipus, condicions de camp i historial d'accions; (2) una arquitectura neuronal (p. ex., xarxa recurrent o encoder + policy network) per captar dependències temporals i la parcialitat de la informació; i (3) estratègies d'entrenament basades en self-play i algorismes actor-crític (com PPO [17]), complementats potencialment amb components d'aprenentatge supervisat per a la predicció de moviments rivals.

L'avaluació es durà a terme en un entorn de simulació local controlat. S'analitzarà el rendiment quantitatiu de l'agent (taxa de victòries i recompensa mitjana acumulada) enfront d'una bateria d'agents basats en regles (scripted baselines) que implementen heurístiques competitives estàndard. Addicionalment, es realitzaran estudis d'ablació (ablation studies) per determinar la contribució de cada mòdul de l'arquitectura. Qualitativament, s'examinarà l'emergència de comportaments avançats com la predicció de canvis de Pokémon (switch prediction) i la gestió del risc, discutint finalment la viabilitat computacional i les limitacions del model proposat.

Paraules clau: Deep Reinforcement Learning (DRL), POMDP, Teoria de Jocs, Sistemes Estocàstics, Pokémon VGC, Xarxes Neuronals.

Chapter 1

Marc Teòric i Contextualització

Aquest capítol estableix els fonaments teòrics necessaris per comprendre el desenvolupament d'un agent d'intel·ligència artificial aplicat a entorns complexos. S'introdueixen els conceptes clau sobre teoria de jocs i IA, es presenta Pokémon i el seu format competitiu VGC com a cas d'estudi, es descriuen les eines de simulació disponibles, i es defineixen les tècniques modernes d'aprenentatge automàtic que sustenten el projecte.

1.1 Contextualització de la IA en Jocs

1.1.1 L'Evolució a través de la Informació Perfecta

Històricament, els jocs han servit com a banc de proves privilegiat per avaluar les capacitats de la Intel·ligència Artificial (IA). Els denominats jocs d'**informació perfecta** — on tots els jugadors coneixen completament l'estat del joc en tot moment — han protagonitzat les fites més emblemàtiques.

Deep Blue (IBM, 1997) va derrotar el campió mundial d'escacs Garry Kasparov emprant l'algorisme *Minimax* amb esporgada alfa-beta [8, 10]. L'estratègia es basava en explorar milions de posicions possibles per torn dins un arbre de cerca, podant les branques que no podien millorar el resultat conegut. En els escacs, aquesta exploració és viable perquè tot l'estat del joc és visible sobre el tauler.

Dues dècades més tard, **AlphaGo** (Google DeepMind, 2016) va aconseguir derrotar Lee Sedol, un dels millors jugadors de Go del món [19]. El Go presenta un factor de ramificació incomparablement superior als escacs (~ 250 moviments legals per torn davant els ~ 35 dels escacs), la qual cosa fa inviable una exploració exhaustiva. Per superar-ho, AlphaGo va combinar la *Cerca d'Arbre de Monte Carlo* (MCTS) [5] amb xarxes neuronals profundes entrenades per *self-play*. La seva evolució, **AlphaZero** [18], va generalitzar l'aproximació per dominar escacs, shogi i Go simultàniament sense coneixement humà previ, aprenent únicament de jugar contra si mateixa.

1.1.2 El Salt als Videojocs Moderns

Les fites anteriors es circumscriuen a jocs de tauler d'informació perfecta i amb regles estàtiques. El gran salt posterior de la IA ha estat conquistar **videojocs moderns**, entorns que es caracteritzen per:

- **Espais d'acció enormes i continus**: centenars d'accions possibles per segon.
- **Informació parcial**: parts del mapa o les accions rivals queden ocultes.
- **Múltiples agents simultanis**: cooperació i competició entre 2–10 jugadors.

- **Horitzons temporals llargs**: partides de 20–60 minuts amb milers de decisions seqüencials.

AlphaStar (DeepMind, 2019) va assolir el nivell Gran Mestre a *StarCraft II*, un joc d'estratègia en temps real d'informació imperfecta. El sistema combinava aprenentatge per imitació sobre partides humanes amb *multi-agent reinforcement learning*, mantenint una lliga d'agents interns que evolucionaven competint entre ells [26].

OpenAI Five (2019) va derrotar el campió mundial del videojoc *Dota 2* en format 5 contra 5. Cada agent controlava un heroi amb més de 170.000 observacions possibles i desenes d'accions per segon, cooperant en equip durant partides de 45 minuts [1].

Fins i tot en jocs d'atzar com el *Pòquer*, sistemes com **Libratus** [3] i **Pluribus** [4] han superat els millors jugadors professionals en la variant *No-Limit Texas Hold'em*, on la informació oculta (les cartes dels rivals) és l'element central de l'estratègia.

1.1.3 Informació Perfecta vs. Informació Imperfecta

Les fites anteriors es poden classificar segons el grau d'observabilitat de l'entorn:

- En un joc d'**informació perfecta** (escacs, Go), l'estat complet del sistema és visible per tots els jugadors en tot moment. Les decisions es basen exclusivament en l'anàlisi de l'estat actual i la predicció de les accions futures del rival.

- En un joc d'**informació imperfecta** (pòquer, StarCraft, Pokémon), part de l'estat roman oculta. Els jugadors han de *deduir* informació a partir d'observacions parcials i gestionar la **incertesa** inherent.

Quan, a més, el resultat de les accions no és determinista sinó que depèn de variables aleatòries (probabilitat de colpejar, dany variable, etc.), es parla d'un entorn **estocàstic**. La combinació d'informació imperfecta i estocasticitat crea entorns extremament desafiants per a la IA.

1.1.4 Processos de Decisió de Markov (MDP i POMDP)

Formalment, la presa de decisions seqüencials en entorns estocàstics es modelitza amb un **Procés de Decisió de Markov (MDP)** [23]. Un MDP estableix que les conseqüències d'una acció depenen únicament de l'estat *actual* del sistema, no de tota la seva història (propietat de Markov). Es defineix com una tupla (S, A, T, R, γ):

- S: Conjunt d'estats possibles.

- A: Conjunt d'accions disponibles.

- T: Funció de transició, $P(s' \mid s, a)$, que indica la probabilitat d'arribar a l'estat s' des de s fent l'acció a.

- R: Funció de recompensa, $R(s, a)$, que quantifica el benefici immediat d'una decisió.

- γ: Factor de descompte ($\gamma \in [0, 1)$), que pondera la importància de les recompenses futures.

Un exemple clàssic d'MDP seria un robot navegant per una quadrícula: coneix la seva posició exacta, les seves accions (moure's en 4 direccions) i les recompenses (arribar a l'objectiu). L'objectiu de l'agent és trobar la **política** $\pi(a \mid s)$ que maximitza la recompensa acumulada esperada.

Tanmateix, si l'agent **no pot observar** l'estat complet del sistema, el model s'estén a un **Procés de Decisió de Markov Parcialment Observable (POMDP)** [20]. Un POMDP afegeix dos elements:

- Ω: Conjunt d'observacions que rep l'agent (una versió parcial o sorollosa de l'estat real).
- O: Funció d'observació, $P(o \mid s', a)$, que condiciona quines observacions arriben a l'agent.

En un POMDP, l'agent construeix i manté un *belief state*: una distribució de probabilitat sobre tots els estats possibles, actualitzada a cada observació rebuda. Per exemple, un jugador de pòquer no veu les cartes dels rivals, però dedueix probabilitats sobre les seves mans a partir de les apostes observades.

La Figura 1.1 il·lustra la diferència fonamental entre ambdós models: mentre que en un MDP l'agent rep l'estat complet del sistema, en un POMDP només accedeix a observacions parcials i ha d'inferir l'estat ocult.

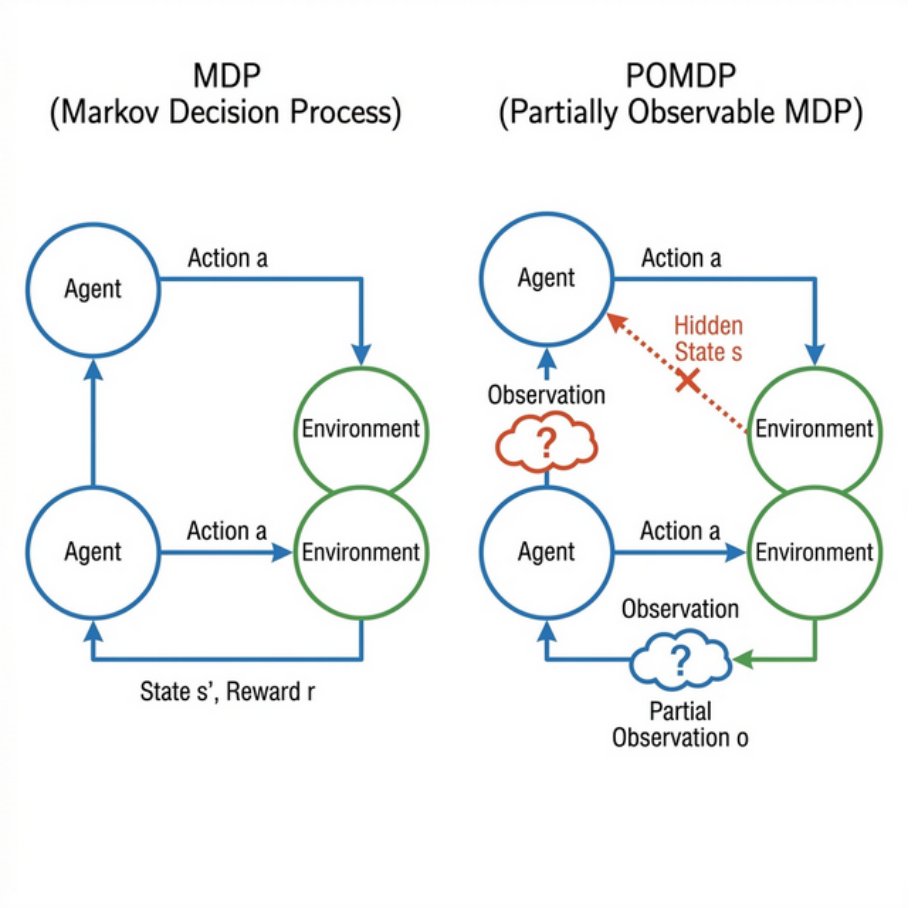

Figure 1.1: Comparació entre un MDP (esquerra) i un POMDP (dreta). En el MDP, l'agent observa l'estat complet s. En el POMDP, l'estat real queda ocult i l'agent només rep observacions parcials o, forçant la construcció d'un *belief state*.

1.1.5 Per què Pokémon és un cas d'estudi ideal?

Pokémon combina de forma natural totes les característiques presentades, convertint-lo en un banc de proves particularment ric per a la IA:

- **Estocasticitat alta**: el dany varia aleatòriament entre el 85% i el 100%, els atacs poden fallar per precisió, i els cops crítics (+50% dany) ocorren amb un ≈4% de probabilitat.
- **Informació imperfecta (POMDP)**: els atacs, estadístiques ocultes, objectes equipats i estratègia del rival no es revelen fins que s'observen empíricament durant la partida.

- **Decisions simultànies**: en el format competitiu, ambdós jugadors decideixen les seves accions al mateix temps, sense conèixer la decisió rival.
- **Espai d'accions combinatori**: especialment en el format de Dobles (2 vs 2), on cal coordinar les accions de dues unitats pròpies contra dues rivals.
- **Joc de suma zero** [27]: el guany d'un jugador és la pèrdua exacta de l'altre.
- **Ecosistema obert**: existeixen simuladors de codi obert i llibreries d'IA que habiliten la investigació a gran escala.

A les seccions següents es detallen les mecàniques del joc, el format competitiu i les eines que fan possible aquest estudi.

1.2 Pokémon: El Joc

1.2.1 Orígens i Franquícia

Pokémon, originat com a contracció del terme japonès *Poketto Monsutā* (*Pocket Monsters*), és la franquícia d'entreteniment més rendible de la història, celebrant el seu 30è aniversari el 2026 (Figura 1.2) [6]. Creat per Satoshi Tajiri i publicat per Nintendo el 1996, el joc original consistia en explorar una regió fictícia amb l'objectiu de capturar criatures, entrenar-les i aconseguir les vuit medalles dels líders de gimnàs per convertir-se en Campió de la Lliga.

Figure 1.2: Logotip commemoratiu del 30è aniversari de la franquícia Pokémon.

El nucli del joc rau en les **batalles per torns** entre Pokémon. Existeixen dos formats principals: **Singles** (1 vs 1, un Pokémon al camp per bàndol) i **Doubles** (2 vs 2, dos Pokémon al camp per bàndol). Sota l'aparença accessible del joc, les batalles amaguen un motor de complexitat matemàtica notable que es detalla a continuació.

1.2.2 Variables de Combat: Estadístiques (Stats)

Cada Pokémon es defineix per sis estadístiques fonamentals que determinen el seu rendiment en combat: Punts de Salut (HP), Atac (físic), Defensa (física), Atac Especial, Defensa Especial i Velocitat. Cada espècie té una distribució base d'stats diferent, especialitzant-la en un rol concret. La Figura 1.3 mostra l'exemple de Pikachu, un Pokémon veloç i ofensiu especialment.

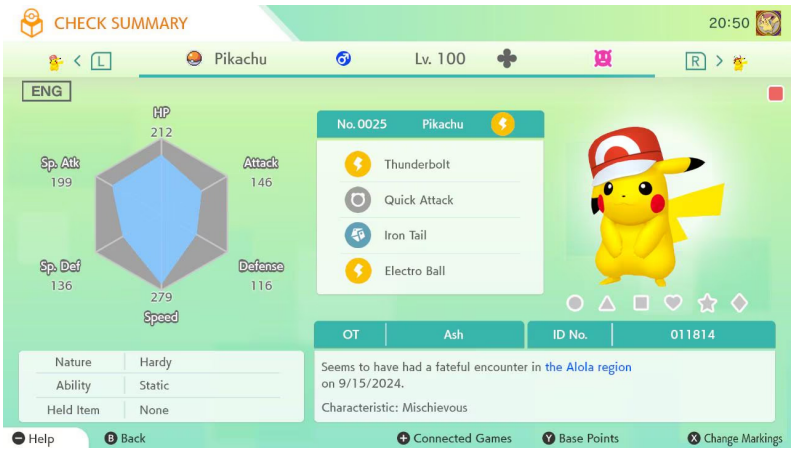

Figure 1.3: Diagrama de les estadístiques base de Pikachu. Es destaquen Velocitat i Atac Especial com a punts forts, amb defenses baixes.

Les estadístiques base es modifiquen per tres factors configurables que constitueixen **informació oculta** durant la batalla:

1. **Individual Values (IVs)**: Valors genètics intrínsecs entre 0 i 31, inherents a cada exemplar. En l'àmbit competitiu, generalment es fixen a 31 per maximitzar el rendiment.

2. **Effort Values (EVs)**: Punts d'entrenament distribuïbles lliurement, amb un límit global de 510 i un màxim de 252 per estadística. Aquesta distribució defineix l'**arquitectura estratègica** del Pokémon i constitueix l'àmbit principal d'engany competitiu, ja que l'oponent desconeix si un Pokémon ha estat construït per atacar, per resistir o per equilibrar-se.

3. **Naturaleses (Natures)**: Modificadors multiplicatius que atorguen un $+10\%$ a una estadística i un -10% a una altra, permetent accentuar el perfil desitjat.

La Figura 1.4 il·lustra la relació entre aquests tres components.

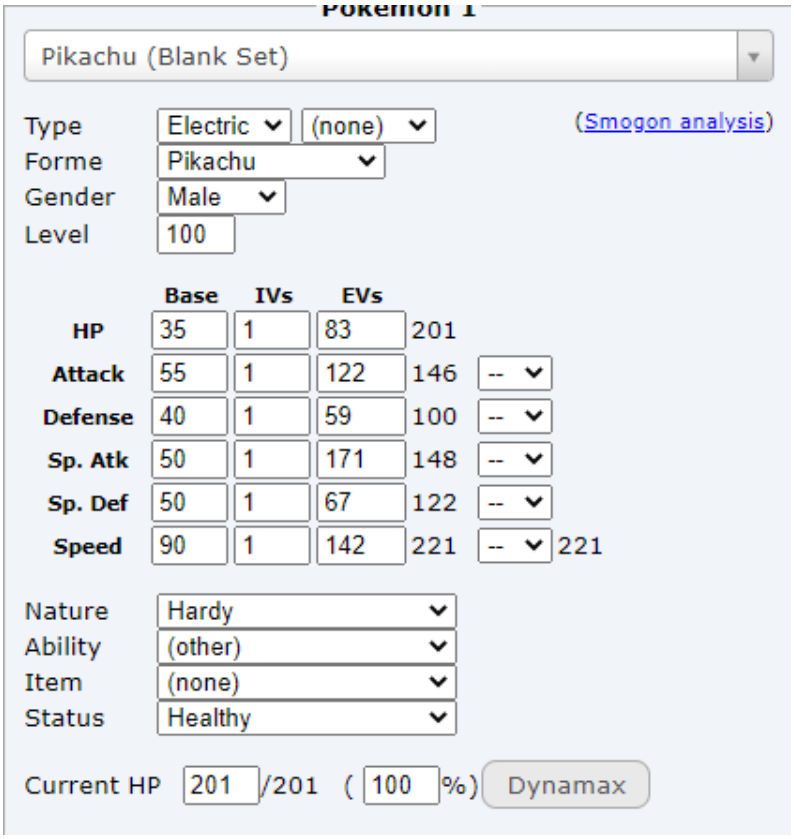

Figure 1.4: Exemple de com IVs, EVs i Nature modifiquen les estadístiques base d'un Pokémon.

El valor final de cada estadística (excepte HP) es calcula amb la fórmula oficial [7]:

$$Stat = \left\lfloor \left(\frac{(2 \times Base + IV + \lfloor \frac{EV}{4} \rfloor) \times Level}{100} + 5 \right) \times Nature \right\rfloor \tag{1.1}$$

Per als Punts de Salut, la fórmula varia lleugerament en afegir una constant extra i no aplicar la Nature:

$$HP = \left\lfloor \frac{(2 \times Base + IV + \lfloor \frac{EV}{4} \rfloor) \times Level}{100} \right\rfloor + Level + 10 \tag{1.2}$$

En el format competitiu, tots els Pokémon es normalitzen a *Level* 50, garantint un camp igualitari.

1.2.3 Taula de Tipus, STAB i Categories d'Atacs

L'eix central de la presa de decisions en combat és la **Taula de Tipus** (Figura 1.5). Existeixen 18 tipus elementals (Aigua, Foc, Planta, Elèctric, etc.) que estableixen una matriu de relacions de dany:

- **Super Effective** ($\times 2$): el tipus de l'atac és fort contra el tipus del defensor.
- **Not Very Effective** ($\times 0.5$): el tipus de l'atac és dèbil contra el defensor.
- **Immunitat** ($\times 0$): el defensor és completament immune.

Com que molts Pokémon posseeixen **doble tipus**, els multiplicadors es combinen, podent generar febleses extremes ($\times 4$) o dobles resistències ($\times 0.25$).

Attacker \ Defender	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Normal													½	0			½	
Fire		½	½	2		2						2	½		½		2	
Water		2	½	½					2				2		½			
Grass		½	2	½				½	2	½		½	2		½		½	
Electric			2	½	½				0	2					½			
Ice		½	½	2		½			2	2					2		½	
Fighting	2					2		½		½	½	½	2	0		2	2	½
Poison				2				½	½				½	½			0	2
Ground		2		½	2			2		0		½	2				2	
Flying				2	½		2					2	½				½	
Psychic							2	2			½					0	½	
Bug		½		2			½	½		½	2			½		2	½	½
Rock		2				2	½		½	2		2					½	
Ghost	0										2			2		½		
Dragon															2		½	0
Dark							½				2			2		½		½
Steel		½	½		½	2							2				½	2
Fairy		½					2	½							2	2	½	

Figure 1.5: Matriu completa de la Taula de Tipus de Pokémon. Cada cel·la indica el multiplicador de dany d'un atac del tipus de la fila contra un defensor del tipus de la columna.

Per il·lustrar-ho amb exemples concrets:

- Un atac *Thunderbolt* (Elèctric) contra un *Squirtle* (Aigua): Elèctric és *Super Effective* contra Aigua → ×2 de dany.
- Un atac *Flamethrower* (Foc) contra un *Bulbasaur* (Planta/Verí): Foc és *Super Effective* contra Planta (×2), però neutre contra Verí (×1), resultant en ×2 total.
- Un atac *Rock Slide* (Roca) contra un *Charizard* (Foc/Volador): Roca és *Super Effective* contra Foc (×2) **i** contra Volador (×2), resultant en una feblesa extrema de ×4 de dany.
- Un atac *Earthquake* (Terra) contra un *Pidgeot* (Normal/Volador): Terra té **immunitat** contra Volador → ×0 de dany, tot i ser normal contra Normal.

Quan un Pokémon utilitza un atac del seu propi tipus, rep el bonus **STAB** (*Same Type Attack Bonus*), un multiplicador addicional de ×1.5 sobre el dany base. Per exemple, quan un *Squirtle* (tipus Aigua) utilitza *Surf* (atac Aigua), el dany es multiplica per ×1.5 automàticament. Combinat amb la Taula de Tipus, un *Surf* amb STAB contra un Pokémon de Foc causaria $1.5 \times 2.0 = \times 3.0$ de dany efectiu.

Els moviments es classifiquen en tres categories:

- **Físics**: utilitzen l'estadística d'Atac de l'atacant contra la Defensa del defensor.
- **Especials**: utilitzen l'Atac Especial contra la Defensa Especial.
- **D'Estat (*Status*)**: no causen dany directe. Permeten estratègies com protegir-se (*Protect*), augmentar estadístiques pròpies (*Buffs*), debilitar l'enemic (*Debuffs*), o infligir condicions d'estat.

En el format de Dobles, existeixen a més atacs **d'àrea (*Spread*)** que colpegen tots els rivals al camp simultàniament, però amb una penalització de dany a ×0.75 [21].

1.2.4 Velocitat i Ordre d'Execució

L'ordre en què els Pokémon actuen cada torn és determinat per l'estadística de **Velocitat**: el Pokémon amb un valor més alt ataca primer. Per exemple, si un *Jolteon* (Velocitat base: 130) s'enfronta a un *Snorlax* (Velocitat base: 30), Jolteon actuarà sempre primer en condicions normals. Controlar la velocitat relativa és fonamental: atacar primer pot significar la diferència entre derrotar l'oponent o ser derrotat abans de poder actuar.

Tanmateix, existeixen rangs de **Prioritat** (de -7 a $+5$) que anul·len completament el càlcul de Velocitat. Un moviment de prioritat $+1$ s'executa sempre abans que qualsevol moviment de prioritat 0, independentment de la Velocitat de qui l'usi. Per exemple:

- *Quick Attack* (prioritat $+1$): Permet a un Pokémon lent com *Snorlax* atacar primer davant d'un *Jolteon* més ràpid, ideal per assegurar un KO contra un oponent amb poca vida.
- *Protect* (prioritat $+4$): S'executa gairebé sempre primer, protegint el Pokémon de tot dany durant un torn. Essencial en el format de Dobles.
- *Trick Room* (prioritat -7): S'executa últim, però inverteix l'ordre de Velocitat durant 5 torns: els Pokémon lents actuen primer. Aquesta mecànica transforma completament les dinàmiques de la partida.

1.2.5 Condicions d'Estat (Status Conditions)

Les condicions d'estat introdueixen disrupcions estocàstiques addicionals al combat, forçant l'agent a adaptar la seva estratègia davant imprevistos. La Taula 1.1 resumeix les principals:

Estat	**Efecte en Stats**	**Mecànica i Exemple**
Cremada (Burn)	$\downarrow$ 50% Atac Físic	Perd 1/16 de la vida per torn. Un *Machamp* cremat perd la meitat de la seva força ofensiva, invalidant el seu rol d'atacant físic.
Paràlisi (Paralysis)	$\downarrow$ 50% Velocitat	25% de probabilitat de no poder actuar cada torn. Un *Jolteon* paralitzat perd el seu avantatge de velocitat i pot quedar incapacitat en moments crítics.
Verí (Poison)	Cap directe	Perd 1/8 de vida per torn; si és verí tòxic, el dany creix exponencialment (1/16, 2/16, 3/16...), penalitzant Pokémon que vulguin mantenir-se al camp molt temps.
Son (Sleep)	Cap directe	El Pokémon no pot actuar durant 1–3 torns aleatoris. Un Pokémon adormit és completament indefens.
Congelació (Freeze)	Cap directe	No pot actuar; 20% de probabilitat de descongelar-se per torn. Potencialment devastador per la seva durada indefinida.

Table 1.1: Condicions d'estat principals i el seu impacte. L'agent ha de considerar tant la probabilitat d'infligir-les com el seu efecte sobre la presa de decisions.

L'impacte estratègic de les condicions d'estat és significatiu: un agent intel·ligent pot optar per infligir una Cremada a l'atacant físic rival en lloc d'atacar directament, reduint la seva amenaça a llarg termini. Igualment, ha de considerar si val la pena substituir un Pokémon enverinat o mantenir-lo al camp per aprofitar un avantatge de tipus.

1.2.6 L'Estocasticitat: El Factor d'Incertesa

Més enllà de les condicions d'estat, el combat Pokémon incorpora múltiples fonts d'**aleatorietat inherent** que impedeixen tota predicció determinista:

1. **Dany Aleatori (*Damage Roll*)**: Cada atac rep un multiplicador aleatori uniforme dins el rang $[0.85, 1.00]$. Això significa que un mateix atac pot causar fins a un 15% menys de dany del previst, convertint KOs aparentment segurs en fallides.

2. **Precisió dels Moviments (*Accuracy*)**: Alguns atacs potents no tenen un 100% de probabilitat d'encertar (p. ex., *Hydro Pump* té un 80% de precisió). L'agent ha de ponderar l'esperança de dany contra el risc de fallar.

3. **Cops Crítics (*Critical Hits*)**: Amb una probabilitat base de $\sim$4.16% (1/24), qualsevol atac pot causar un 50% més de dany, ignorant les modificacions defensives de l'oponent [21].

Aquesta triple font d'aleatorietat fa que **cap estratègia garanteixi un resultat al 100%**: la millor heurística del món pot perdre contra un rival inferior si l'atzar li és desfavorable. Aquesta característica defineix el marge residual irreductible de victòria del qual cap IA podrà escapar.

1.3 El Format Competitiu: VGC

1.3.1 Orígens i Història

El circuit **VGC** (*Video Game Championships*) va ser creat per *The Pokémon Company* l'any 2009 com a format oficial de competició a nivell mundial [24]. Des d'aleshores, centenars de milers de jugadors hi participen anualment en tornejos regionals, nacionals i el *Pokémon World Championships*, amb premis de fins a 30.000$ per al campió [25].

Figures com **Wolfe Glick** (WolfeyVGC), doble campió del món (2016 i 2019), han contribuït a divulgar l'escena competitiva a través de contingut educatiu accessible [9]. La comunitat d'estratègia competitiva, amb plataformes com *Smogon University* [22], també ha desenvolupat durant dècades un cos ingent d'anàlisi teòrica sobre el metagame.

1.3.2 En què es diferencia del Pokémon "normal"

El VGC es diferencia del mode Adventure del joc original en diversos aspectes fonamentals:

1. **Format de Dobles (2 vs 2)**: A diferència de les batalles Singles del mode Adventure, el VGC oficial és exclusivament de **Dobles**: cada jugador té dos Pokémon simultàniament al camp de batalla (Figura 1.6). Això multiplica l'espai de decisions i introdueix conceptes com la sinergia entre aliats, els atacs d'àrea, i la protecció (*Protect*).

2. **Team Preview i Bring 6, Pick 4**: Abans de cada partida, ambdós jugadors veuen els 6 Pokémon de l'equip rival i n'escullen 4 per competir. Aquesta fase de selecció és en si mateixa un joc d'informació imperfecta basat en la predicció.

3. **Decisió simultània**: Ambdós jugadors escullen les accions dels seus dos Pokémon al *mateix temps*, sense conèixer la decisió rival. Això obliga a preveure les accions de l'oponent.

4. **Regulació per temporades**: El VGC canvia les regles cada temporada i cada generació de jocs, redefinint quins Pokémon són legals i quines mecàniques s'apliquen. Això genera un metagame en constant evolució.

5. **Restriccions (Clauses)**: La *Species Clause* (no repetir espècies) i la *Item Clause* (no repetir objectes) forcen diversitat estratègica als equips.

Figure 1.6: Captura d'un combat de Dobles VGC al simulador Pokémon Showdown. S'observen els quatre Pokémon al camp (dos per bàndol), les barres de vida, les condicions actives i l'efecte climàtic.

1.3.3 El Metagame i les Generacions

El terme **metagame** ("meta") fa referència a les estratègies dominants en un moment donat. Cada nova generació de jocs de Pokémon (de la Generació I a la IX actual) introdueix noves espècies, moviments, objectes i mecàniques que transformen radicalment les estratègies viables.

Per exemple, la *Generació VIII* (Pokémon Sword/Shield) va introduir el *Dynamax*, una mecànica que permetia duplicar la vida d'un Pokémon temporalment, mentre que la *Generació IX* (Pokémon Scarlet/Violet) va substituir-la per la *Teracristal·lització*, que permet canviar el tipus d'un Pokémon en ple combat. Cadascuna generació presenta un entorn de complexitat diferent amb el qual l'agent d'IA hauria de poder operar.

Comunitats com *Smogon* [22] mantenen una classificació organitzada de Pokémon per tiers d'ús (OU, UU, RU, etc.) i compilen guies estratègiques que defineixen el meta de cada generació i format, sent un recurs valuós tant per a jugadors humans com per a investigadors.

1.4 L'Entorn Virtual: Eines de Simulació

La naturalesa tancada dels jocs de consola originals impedeix la manipulació directa de les batalles per part d'un agent d'IA. Per això, la comunitat ha desenvolupat eines de codi obert que habiliten la investigació. Dues eines fonamentals per a aquest projecte:

1.4.1 Pokémon Showdown

Pokémon Showdown [13] és el simulador de batalles competitives més utilitzat del món. Desenvolupat en JavaScript/TypeScript, replica amb precisió mil·limètrica totes les mecàniques del joc oficial (fórmules de dany, taula de tipus, IVs/EVs, condicions d'estat, etc.) mitjançant enginyeria inversa dels jocs originals.

La seva característica clau per a la investigació rau en que pot funcionar com a **servidor local** sense interfície gràfica, permetent executar milers de batalles per minut de forma automatitzada. Això el converteix en un entorn de simulació ideal per entrenar agents d'IA.

1.4.2 Poke-env

Poke-env [16] és una llibreria de Python que actua com a capa d'abstracció (*API wrapper*) sobre Pokémon Showdown. Tradueix el flux de comunicació del servidor (missatges WebSocket) a objectes Python estructurats, proporcionant:

- Representació de l'estat de la batalla com a objectes accesibles (Pokémon al camp, vida restant, moviments disponibles, etc.).
- Interfície estàndard compatible amb l'estructura d'*OpenAI Gym* [2] per definir agents de *Reinforcement Learning*.
- Agents de referència (*baselines*) integrats per a validació i comparació.

La combinació de Showdown com a motor i Poke-env com a interfície estableix l'ecosistema complet que habilita aquest TFM: un entorn reproduïble, escalable i fidel a les regles oficials del joc. La seva configuració tècnica detallada es presenta al Capítol 2.

1.5 Intel·ligència Artificial i Tècniques Computacionals

Aquesta secció presenta les tècniques algorísmiques i d'aprenentatge automàtic rellevants per al projecte, des dels mètodes clàssics de cerca fins a les arquitectures neuronals modernes.

1.5.1 Algorismes de Cerca Clàssics

Minimax amb Esporgada Alfa-Beta. L'algorisme *Minimax* [10] modela un joc de dos jugadors com un arbre de decisions on un jugador (*Max*) intenta maximitzar la seva puntuació i l'altre (*Min*) intenta minimitzar-la. Explora totes les branques possibles fins a una profunditat determinada, assignant valors als estats terminal i propagant-los cap enrere. L'**esporgada alfa-beta** redueix el nombre de nodes explorats descartant branques que no poden millorar el resultat ja trobat.

Limitació: El Minimax funciona bé en jocs amb un factor de ramificació moderat (escacs: ~ 35), però col·lapsa en entorns estocàstics d'alta dimensionalitat com Pokémon, on la combinació de variables ocultes i aleatorietat fa inviable l'exploració exhaustiva.

Cerca d'Arbre de Monte Carlo (MCTS). L'MCTS [5] supera les limitacions del Minimax emprant simulacions aleatòries per estimar el valor d'un estat. El procés iteratiu segueix quatre fases:

1. **Selecció**: Des de l'arrel, selecciona el node fill més prometedor (balançant exploració i explotació amb UCB1).
2. **Expansió**: Afegeix un nou node fill al node seleccionat.
3. **Simulació (*Rollout*)**: Des del nou node, es juga una partida aleatòria fins al final.
4. **Retropropagació**: El resultat de la simulació s'actualitza cap amunt a tots els nodes predecessors.

Repetint milers de vegades aquest cicle, l'MCTS convergeix cap a una estimació robusta de quina acció és millor, sense necessitat d'explorar tot l'arbre. Va ser clau en l'èxit d'AlphaGo [19].

1.5.2 Xarxes Neuronals Artificials

Les xarxes neuronals [11] són funcions d'aproximació universals compostes per capes de neurones interconnectades. Cada neurona aplica una transformació lineal seguida d'una funció d'activació no lineal. Mitjançant l'*aprenentatge* (ajust dels pesos per *backpropagation* sobre dades), una xarxa pot aprendre a reconèixer patrons complexos.

Les arquitectures més rellevants per a jocs inclouen:

- **Xarxes Denses (MLP)**: Capes totalment connectades, adequades per processar vectors d'estat de dimensió fixa.
- **Xarxes Recurrents (LSTM/GRU)**: Incorporen memòria temporal, útils per capturar seqüències d'accions passades i deduir l'estat ocult en POMDPs.
- **Xarxes Convolucionals (CNN)**: Especialitzades en dades espacials, menys rellevants en el nostre cas.

1.5.3 Reinforcement Learning (RL)

El **Reinforcement Learning** (Aprenentatge per Reforç) [23] és un paradigma d'aprenentatge automàtic on un **agent** interactua amb un **entorn** de forma iterativa: observa l'estat, executa una acció, rep una recompensa, i repeteix. L'objectiu és aprendre una política π que maximitzi la recompensa acumulada a llarg termini.

El RL es diferencia de l'aprenentatge supervisat en un aspecte crucial: l'agent **no disposa d'exemples correctes** (*labels*), sinó que ha de **descobrir** quines accions són bones mitjançant l'experiència pròpia (prova i error).

Els mètodes de RL es classifiquen en:

- **Value-based** (basat en valor): L'agent aprèn una funció $Q(s, a)$ que estima el valor de fer una acció a en l'estat s. Exemple: DQN.
- **Policy-based** (basat en política): L'agent aprèn directament la política $\pi(a \mid s)$ com a distribució de probabilitat sobre les accions.
- **Actor-Critic**: Combina ambdós: una xarxa *Actor* aprèn la política i una xarxa *Critic* avalua com de bona és cada acció. Exemple: PPO.

DQN (Deep Q-Network)

Introduït per DeepMind [14], DQN aproxima la funció $Q(s, a)$ amb una xarxa neuronal profunda. Dues innovacions clau:

- **Experience Replay**: Emmagatzema les experiències (s, a, r, s') en un buffer i entrena sobre mostres aleatòries, trencant la correlació temporal.
- **Target Network**: Utilitza una còpia fixa de la xarxa Q per calcular els objectius d'entrenament, estabilitzant l'aprenentatge.

DQN funciona bé amb espais d'acció discrets i moderats, com és el cas de les batalles Pokémon (on les accions legals per torn són un conjunt finit d'atacs i substitucions).

PPO (Proximal Policy Optimization)

PPO [17] és un algorisme Actor-Critic desenvolupat per OpenAI que ha demostrat un rendiment molt robust en entorns complexos. Intenta resoldre el problema central del RL basat en política: actualitzacions massa grans que destrueixen l'aprenentatge acumulat. Ho aconsegueix amb la

tècnica de *Clipping*: restringeix la magnitud de cada actualització dins d'un rang $[1-\epsilon, 1+\epsilon]$, evitant canvis bruscos.

Aquesta estabilitat és particularment valuosa en entorns **altament estocàstics com Pokémon**, on una recompensa enganyosament alta (p. ex., guanyar per un cop crític afortunat) podria desestabilitzar completament l'entrenament sense la protecció del clipping.

1.5.4 Pokechamp i PokeLLMon: Agents amb Models de Llenguatge

Pokechamp [12] (*PokéChamp: an Expert-level Minimax Language Agent*, ICML 2025) és un repositori de codi obert que combina dues tècniques presentades anteriorment — Minimax i Models de Llenguatge de Gran Escala (LLMs) — per crear un agent de nivell expert en batalles Pokémon. Pokechamp utilitza un LLM per raonar sobre l'estat de la batalla (avaluar posicions, predir moviments del rival) i combina aquest raonament amb una cerca Minimax d'un pas per seleccionar l'acció òptima.

El repositori inclou diversos agents amb aproximacions diferents:

- **Pokechamp (LLM + Minimax)**: L'agent principal. Utilitza un LLM per generar avaluacions heurístiques dels estats i una cerca Minimax guiada per optimitzar la decisió final.
- **PokeLLMon (LLM)**: Un agent alternatiu basat exclusivament en LLM, que pren decisions directament a partir de la descripció textual de la batalla sense cerca addicional.
- **Abyssal (Heurístic)**: Agent basat en regles que avalua matchups de tipus i estima el dany.
- **OneStep (Heurístic)**: Agent amb estimació de dany a un torn vista (*one-step lookahead*).

Els **LLMs** (*Large Language Models*) són xarxes neuronals de gran escala entrenades sobre corpus massius de text que exhibeixen capacitats de raonament, generació de codi i anàlisi estratègica. En el context de Pokémon, permeten a l'agent "llegir" l'estat de la batalla en text natural i "raonar" sobre quina acció prendria un jugador expert. La seva capacitat per processar informació parcial i fer inferències el fa un mètode prometedor per abordar POMDPs.

Pokechamp serveix en aquest TFM com a font de *baselines* addicionals — tant heurístics com basats en LLM — contra els quals comparar el rendiment dels agents desenvolupats, complementant els agents natius de poke-env. Els detalls d'implementació i configuració dels LLMs s'exposen al Capítol 3.

1.6 Justificació de la Recerca

El propòsit d'investigar el competitiu de Pokémon no és crear un bot imbatible per entreteniment, sinó utilitzar aquest ecosistema com un **laboratori rigorós** per validar les capacitats i limitacions d'arquitectures d'IA autònoma.

Pokémon VGC sintetitza reptes representatius de problemes reals d'enginyeria i gestió on un agent ha d'operar sota:

- **Soroll estocàstic alt**: les decisions òptimes poden resultar en fracàs per atzar, amb una taxa irreductible de pèrdues.
- **Informació parcial (POMDP)**: l'agent ha de deduir informació oculta a partir d'observacions limitades.
- **Adversaris intel·ligents i adaptatius**: l'oponent humà (o agent) modifica la seva estratègia dinàmicament.

Construir agents que evolucionin des d'heurístiques preprogramades fins a models d'aprenentatge autònom permet validar la robustesa dels sistemes de decisió en entorns complexos. La disponibilitat d'eines obertes (Showdown, poke-env, Pokechamp) i la possibilitat de realitzar milers d'experiments controlats fan de Pokémon una plataforma excepcionalment adequada per a la recerca acadèmica en IA.

Chapter 2

Metodologia: Disseny Experimental

Per poder dissenyar, avaluar i entrenar agents d'IA en el metagame Pokémon presentat al capítol anterior, cal estructurar les bases experimentals del projecte. Aquest capítol detalla l'entorn de simulació, la complexitat del problema, els espais formals de treball i les mètriques d'avaluació. Finalment, s'exposa l'estratègia de desenvolupament en fases progressives.

2.1 Entorn de Simulació

2.1.1 Arquitectura Interna de Pokémon Showdown

Pokémon Showdown [13] s'executa com un servidor Node.js que processa la lògica del combat de forma determinista donats uns inputs. El flux de comunicació funciona mitjançant **WebSockets**:

1. El servidor envia l'estat observable de la batalla en format text (vida dels Pokémon, moviments disponibles, condicions actives, etc.).
2. Cada jugador respon amb una ordre codificada (`/choose move 1`, `/choose switch 3`, etc.).
3. El servidor resol el torn aplicant les regles oficials (prioritat, velocitat, RNG) i envia el nou estat als jugadors.

L'avantatge clau per a la investigació és que Showdown pot funcionar com a **servidor local sense interfície gràfica**. Això permet:

- Executar batalles a gran velocitat (centenars per minut).
- Controlar completament les variables d'entorn (format, generació, equips inicials).
- Automatitzar la recopilació de dades massives per a entrenament d'agents.

2.1.2 Poke-env com a Interfície Python

La llibreria poke-env [16] tradueix els missatges WebSocket del servidor a objectes Python estructurats. Proporciona:

- **Classe** `Battle`: Conté tot l'estat observable — Pokémon actius, vida restant, moviments disponibles, condicions de camp, registre d'accions passades — com a atributs directament accesibles.
- **Classe** `Player`: Classe base per definir agents. El mètode `choose_move(battle)` rep l'estat i retorna l'acció seleccionada.

- **Compatibilitat Gym**: Proporciona interfície compatible amb l'estàndard *OpenAI Gym* [2], habilitant l'ús directe d'algorismes de RL com DQN o PPO.
- **Gestió asíncrona**: Tota la comunicació es gestiona amb `asyncio` de Python, permetent concurrència nativa.

2.1.3 Variables d'Estat Disponibles

L'objecte `Battle` que rep l'agent cada torn exposa les següents variables observables:

Variable	**Descripció**
HP propis	Percentatge i valor absolut de vida de cada Pokémon propi (actius i reserva).
HP rivals	Percentatge de vida dels Pokémon rivals observats al camp.
Moviments propis	Llista de fins a 4 moviments per Pokémon: nom, tipus, potència, precisió, PP restants.
Moviments rivals coneguts	Moviments observats de l'oponent durant la batalla (informació parcial acumulativa).
Tipus de Pokémon	Tipus primari i secundari de cada Pokémon al camp.
Boosts	Modificadors d'estadístiques actius (↑/↓ en Atac, Defensa, Velocitat, etc.).
Condicions (Status)	Estats alterats actius: Burn, Paralysis, Sleep, Poison, Freeze.
Condicions de Camp	Efectes globals: clima (Sol, Pluja, etc.), terrenys, hazards (*Stealth Rock*, etc.).
Objectes	Objecte equipat dels Pokémon propis; l'objecte rival queda ocult fins que s'activa.
Habilitats	Habilitat passiva de cada Pokémon; la rival es pot deduir pel seu efecte.
Team Preview	Composició dels 6 Pokémon rivals (només espècies, sense stats ni objectes).

Table 2.1: Variables d'estat observables per l'agent a cada torn a través de poke-env.

Informació oculta: En tot moment, l'agent **desconeix** les EVs, IVs, Nature, moviments no revelats, objecte i habilitat dels Pokémon rivals, així com l'ordre dels Pokémon que l'oponent manté a la reserva. Això confirma formalment que l'entorn és un POMDP.

2.1.4 Infraestructura Paral·lela de Servidors

L'execució de benchmarks estadísticament significatius (centenars de batalles per parella d'agents) amb un sol servidor resulta massa lenta. Per solucionar-ho, s'ha dissenyat una infraestructura d'orquestració pròpia (situada a `src/p03_scripts`) que permet:

1. **Llançament massiu de servidors**: Aixecar dinàmicament múltiples instàncies de Showdown en ports consecutius (ex. 8000, 8001, 8002...), deshabilitant protocols de *login* innecessaris.
2. **Distribució d'agents**: El codi orquestrador Python llança tasques asíncrones massives i distribueix les batalles entre els ports disponibles.
3. **Gestió de memòria**: Neteja manual del Garbage Collector després de cada batch de partides per evitar fuites de memòria dels processos Node.js.
4. **Checkpoints de represa**: Emmagatzematge en fitxers JSON del progrés de cada parella d'agents, permetent reprendre l'execució si s'interromp.

Aquesta infraestructura permet executar centenars de batalles simultànies, cosa que resulta vital tant per al benchmarking heurístic com per a la futura recopilació d'experiència dels algorismes de RL.

2.2 Complexitat del Problema

2.2.1 Diferencies entre Generacions

Cada generació de jocs de Pokémon defineix un conjunt diferent de regles, espècies disponibles i mecàniques especials. Això impacta directament en la complexitat del problema:

- **Nombre de Pokémon legals**: Varia entre ∼150 (Generació I) i >1.000 (Generació IX).
- **Mecàniques especials**: Mega-Evolucions (Gen VI–VII), Z-Moves (Gen VII), Dynamax (Gen VIII), Teracristal·lització (Gen IX) afegeixen accions extra i modificadors.
- **Objectes i habilitats**: La base creix a cada generació, augmentant les combinacions possibles.

Per gestionar aquesta variabilitat, l'arquitectura del projecte s'ha dissenyat per ser flexible: el format i la generació es passen com a paràmetres configurables, permetent entrenar i avaluar agents en qualsevol combinació.

2.2.2 Singles (1 vs 1) vs. Doubles (2 vs 2)

La diferència de complexitat entre formats és considerable:

Dimensió	**Singles (1v1)**	**Doubles (2v2)**
Pokémon al camp	1 propi, 1 rival	2 propis, 2 rivals
Accions per torn	≤ 9 (4 atacs + 5 switches)	$\leq 9 \times 9 = 81$ combinacions
Objectius d'atac	1 (el rival)	3 (rival 1, rival 2, aliat)
Sinergia	No aplica	Crític (proteccions, atacs spread, suport)
Decisions simultànies	2 (1 per jugador)	4 (2 per jugador)

Table 2.2: Comparació de la complexitat dels formats Singles i Doubles.

El format de **Doubles** multiplica exponencialment l'espai de decisions: cada jugador ha de coordinar les accions de dos Pokémon simultàniament, tenint en compte sinergies, atacs d'àrea, proteccions i l'interacció entre els quatre Pokémon al camp. Per aquesta raó, el projecte aborda primer el format Singles com a prova de concepte i després escala al format Doubles.

2.3 Espai d'Estats i Espai d'Accions

Tot problema de RL ha d'estar encapsulat formalment per poder ser processat per un algorisme.

2.3.1 Espai d'Estats (S)

El conjunt S agrupa tota la informació observable per l'agent en un torn donat. Això constitueix un entorn parcialment observable (POMDP), on l'agent només accedeix a una observació $\mathbf{O}_t \in \mathbb{R}^d$ que resumeix:

- Vida (HP) restant de tots els Pokémon visibles (propis i rivals).
- Modificadors d'estadístiques actius (*boosts*).

- Condicions d'estat (status) actives.
- Informació de camp (clima, terreny, hazards).
- Historial acumulat de moviments i tipus observats del rival.

En el format de Singles, el vector d'observació resultant té de l'ordre de $d \sim 50$–100 dimensions. En Doubles, es duplica aproximadament per la presència de dos Pokémon actius per bàndol.

2.3.2 Espai d'Accions (A)

L'espai d'accions A conté els moviments i substitucions legals disponibles:

- **Singles**: Fins a 4 atacs + fins a 5 switches = màxim 9 accions discrets per torn.
- **Doubles**: L'agent ha d'escollir una acció per a *cadascun* dels dos Pokémon al camp. A més, cada atac ha d'especificar l'objectiu (*target*): rival 1, rival 2, o aliat. Això genera un producte cartesià de possibilitats que pot arribar a desenes de combinacions legals per torn.

Cal notar que l'espai d'accions és **dinàmic**: varia a cada torn segons els Pokémon vius, els PP restants, si un Pokémon ha estat derrotat i obliga un switch forçós, etc.

2.4 Mètriques d'Avaluació

La naturalesa estocàstica del combat Pokémon invalida qualsevol avaluació basada en poques partides: un sol resultat pot estar dominat per l'atzar. Per tant, totes les comparacions entre agents es realitzen sobre sèries massives de batalles (≥ 100 per parella).

2.4.1 Mètriques Primàries

- **Win-Rate (%)**: Percentatge de victòries de l'agent sobre el total de batalles disputades contra un oponent concret. És la mètrica principal i definitiva del projecte. Referència interpretativa:
  - $\sim 50\%$: L'agent no millora respecte un rival aleatori simètric.
  - $> 60\%$: Avantatge significatiu; l'agent demostra intel·ligència estratègica.
  - $> 85\%$: Domini clar; l'agent supera àmpliament l'oponent.
  - 100%: Intrínsecament impossible en un entorn estocàstic com Pokémon.

2.4.2 Mètriques Secundàries

Per complementar el Win-Rate i obtenir una visió més granular del comportament dels agents, es recullen mètriques addicionals:

- **Durada de les partides (torns)**: Un agent eficient tendeix a guanyar en menys torns. Partides molt llargues poden indicar indecisió o incapacitat d'aprofitar avantatges.
- **Pokémon derrotats (propis)**: Quantes unitats ha perdut l'agent durant la partida. Un nombre baix indica millor gestió defensiva.
- **Pokémon derrotats (rivals)**: Quantes unitats rivals ha eliminat l'agent. Pot diferenciar entre victòries ajustades i dominants.
- **Pokémon restants vius**: Tant propis com rivals al final de la partida. Indica el marge de victòria o derrota.

- **Vida total restant (%)**: Suma de la vida percentual de tots els Pokémon supervivents (propis i rivals). Aporta una mesura contínua i més granular que el simple recompte de Pokémon vius.

2.5 Fases de Desenvolupament i Models de Referència

El desplegament d'agents sobre *Showdown* s'ha estructurat metodològicament en dues grans fases consecutives per garantir una monitorització i isolació de variables controlada.

2.5.1 Fase 1: Models de Regles Expertes (Heurístiques)

Abans de submergir el codi en hiperparàmetres de *Machine Learning*, la prioritat metodològica ha estat dissenyar models **heurístics propis** basats en regles de decisió expertes (arbres "If-Else" avançats). Aquest acostament primari ha complert un triple propòsit fonamental:

- **Validació de la Infraestructura**: Funcionar com a prova de concepte de l'entorn (PoC) per garantir que el nostre orquestrador de servidors asíncrons llegia i enviava decisions sense caigudes de memòria (*Memory Leaks* de Node.js) processant milers d'estats per minut.
- **Depuració de la Lògica Numèrica**: Comprovar físicament que l'accés a les variables d'estat de *poke-env* (HP, clima, efectes secundaris actius) convergia en comportaments teòrics òptims.
- **Desenvolupar el Límit de Control Propi (Baseline Suprem)**: Mitjançant l'evolució iterativa d'aquestes heurístiques, el projecte ha consolidat un "Límit de Control Expert". L'objectiu últim serà llançar futurs agents neuronals contra aquesta heurística madura, forçant la xarxa neuronal a superar raonament matemàtic tancat creat per disseny expert.

2.5.2 Fase 2: Models d'Aprenentatge Profund

Una vegada validada l'arquitectura i disposant del *Baseline Expert*, la segona gran fase de la recerca serà la substitució de la codificació dura per arquitectures d'aprenentatge. L'eix científic del TFM assoleix el seu màxim pragmatisme avaluant si algorismes com DQN o PPO poden descobrir per si sols les normes causals que la Fase 1 programava explícitament (Taula de Tipus, STAB, predicció de canvis).

2.5.3 Línies de Control (Baselines)

Amb l'entorn escalable definit, cal fixar les línies base de referència sobre les quals mesurar tot progrés. S'utilitzen dos conjunts de baselines complementaris.

Baselines natius de poke-env. Dins de poke-env [16] es proporcionen tres agents de referència que funcionen en **qualsevol generació i format** (Singles i Doubles):

1. ***RandomPlayer***: Efectua moviments a l'atzar entre les accions legals. Actua com a referència mínima: qualsevol agent amb un mínim d'intel·ligència hauria de superar-lo àmpliament.
2. ***MaxBasePowerPlayer***: Un agent cobdiciós (*Greedy*) que escull sempre l'atac amb la potència base més alta, ignorant tipus, STAB i context defensiu. Representa la barrera inferior d'estratègia funcional.

3. ***SimpleHeuristicPlayer***: El rival més sofisticat ofert per la llibreria. Incorpora heurístiques preprogramades: avalua debilitats de tipus, prioritza canvis (*switches*) quan el seu Pokémon actiu es troba en desavantatge, i maximitza el dany considerant matchups. Superar aquest agent de manera consistent certifica que les noves funcions pròpies han assolit una intel·ligència superior.

Baselines externs de Pokechamp. Des del repositori Pokechamp [12] s'han integrat agents addicionals, principalment orientats al format **Singles de Generació 9** (`gen9randombattle`):

1. ***AbyssalPlayer***: Agent heurístic avançat que avalua matchups de tipus, estima el dany esperat i decideix substitucions basant-se en l'anàlisi de tot l'equip rival. Incorpora lògica específica de Gen 9 (Teracristal·lització). Funciona com a rèplica millorada del SimpleHeuristicPlayer amb coneixement generacional.

2. ***SafeOneStepPlayer***: Agent amb estimació de dany a un torn vista (*one-step lookahead*): puntua cada moviment per potència $\times$ STAB $\times$ efectivitat $\times$ precisió i escull el millor. S'ha reimplementat internament per evitar dependències del simulador local de Pokechamp que causaven bloquejos.

3. ***Pokechamp i PokeLLMon (LLM)***: Agents basats en Models de Llenguatge (descrits al Capítol 1) que utilitzen raonament textual per prendre decisions. Requereixen un backend LLM local (detalls d'implementació al Capítol 3). Aplicables exclusivament a **Singles Gen 9**.

El contrast sistemàtic contra **ambdós conjunts** de baselines — natius i externs — constitueix el filtre fonamental de la metodologia, a través del qual qualsevol avanç real serà rigorosament validat.

Chapter 3

Implementació d'Agents: De les Fases Heurístiques a l'Aprenentatge Profund

D'acord amb la metodologia evolutiva exposada al capítol anterior, aquest apartat detalla l'arquitectura de software construïda per donar vida als agents racionals dins del simulador *Pokémon Showdown*. Es cobreixen la infraestructura d'orquestració, l'evolució de les heurístiques, la integració d'agents externs i LLMs, i la implementació del pipeline de Reinforcement Learning.

3.1 Orquestració Operacional i Entorn Paral·lelitzat

El motor d'aquest projecte recau en un mòdul orquestrador propi, situat a `src/p03_scripts/`, encarregat de trencar la limitació original d'un sol fil de connexió pròpia de *poke-env*. Fent ús intensiu de l'asincronia de Python (`asyncio`), els scripts desenvolupats són capaços de:

1. **Llançament massiu de servidors Showdown**: Aixecar dinàmicament múltiples instàncies de Pokémon Showdown en ports consecutius (p. ex. 8000, 8001, 8002...), cadascuna com a procés Node.js independent. Es deshabiliten serveis innecessaris (*verifier*, *friends*, *repl*) per reduir consum i prevenir errors de connexió.

2. **Distribució d'agents**: L'orquestrador Python distribueix les batalles entre els ports disponibles mitjançant tasques asíncrones massives amb `asyncio`, permetent que múltiples batalles s'executin simultàniament.

3. **Gestió de memòria**: Neteja manual del Garbage Collector de Python a l'acabar petits *batches* de partides. A més, els servidors Showdown es reinicien automàticament cada cert nombre de matchups (`--restart-every`) per evitar l'acumulació de processos `room-battle.js` de Node.js que no s'alliberen.

4. **Checkpoints de represa**: Estructures JSON al directori `data/` registren el progrés de cada parella d'agents, permetent reprendre l'execució si s'interromp per qualsevol motiu.

3.2 Fase 1.1: Desenvolupament d'Heurístiques per Singles (1vs1)

Abans de desplegar models d'aprenentatge, el projecte exigeix validar la infraestructura dins d'un espai d'accions reduït. El format **Singles** (1vs1) serveix com a arena de desenvolupament: l'agent interactua contra un sol oponent visible, amb un màxim de 9 accions per torn (4 atacs + 5 substitucions).

Mitjançant sis cicles evolutius de codi, s'han consolidat heurístiques progressivament més sofisticades sota `src/p01_heuristics/s01_singles/agents/internal/`:

- **Heurística v1 (Naïve Aggressive)**: Selecciona el moviment amb la potència base més alta, ignorant completament tipus, STAB i substitucions.
- **Heurística v2 (Damage Awareness)**: Introdueix el càlcul de dany estimat, multiplicant la potència base pel factor d'efectivitat de tipus ($\times 0.5$, $\times 1$, $\times 2$, $\times 4$) i l'STAB ($\times 1.5$).
- **Heurística v3 (Defensive Pivot)**: Afegeix la primera lògica defensiva. Si el Pokémon actiu està enverinat (Toxic) durant més de 2 torns, es substitueix. Si l'oponent és més ràpid i l'atac previst fa menys de 20 HP de dany, es busca un company millor. Aquesta lògica de *switching* és tan estable que les versions posteriors (v4–v6) la reutilitzen.
- **Heurística v4 (Field-Aware)**: Refina el càlcul de dany incorporant el sistema de categories (físic vs. especial, considerant si l'atacant està cremat) i els modificadors climàtics i de terreny (Sol, Pluja, Electric/Grassy/Psychic Terrain).
- **Heurística v5 (Boost-Aware)**: Estén v4 comptant les modificacions d'estadístiques en combat (*boosts*): considera els $+/-$ d'Atac, Defensa i Velocitat en la fórmula de dany, i afegeix regles de seguretat (substitucions preventives quan la vida pròpia és baixa).
- **Heurística v6 (Priority Master)**: Integra totes les regles anteriors i afegeix un multiplicador de $\times 1.2$ als moviments de prioritat (*Quick Attack*, *Shadow Sneak*, etc.), permetent assegurar KOs contra oponents amb poca vida abans de rebre un atac.

3.3 Fase 1.2: Integració d'Agents Externs (Pokechamp)

Per ampliar el conjunt de baselines i disposar d'agents externs contra els quals validar les heurístiques, s'ha integrat el repositori **Pokechamp** [12] al pipeline de benchmarking.

3.3.1 Agents Heurístics de Pokechamp (1vs1)

S'han incorporat els agents basats en regles de Pokechamp per al format Singles Generació 9:

- **AbyssalPlayer**: Agent heurístic que combina avaluació de matchups de tipus, estimació de dany amb STAB i efectivitat, i anàlisi de l'equip rival complet. Incorpora lògica específica de la Generació 9, incloent la Teracristal·lització.
- **SafeOneStepPlayer**: Reimplementació pròpia de l'agent *OneStepPlayer* original de Pokechamp. L'original utilitzava un simulador local (*LocalSim*) que causava bloquejos (*hangs*) per dependències de fitxers de cache absents. La nostra versió calcula dany = potència $\times$ STAB $\times$ efectivitat $\times$ precisió usant exclusivament l'API de poke-env, sense dependències externes.

3.3.2 Agents LLM: Pokechamp i PokeLLMon

Els agents basats en Models de Llenguatge (LLMs) representen un enfocament radicalment diferent: en lloc de regles codificades, utilitzen raonament en text natural per escollir accions.

Backend: Ollama amb Qwen3 8B. Per executar els LLMs de forma local sense dependre d'APIs costoses, s'ha configurat **Ollama** com a servidor d'inferència local. El model seleccionat, **Qwen3 8B**, ocupa ∼6–8 GB de VRAM i encaixa completament dins d'una GPU RTX 2080 (8 GB), permetent una inferència ràpida (∼2–5 segons per torn). Models més grans (13B+) causaven *memory swapping* que alentia cada torn a minuts.

La configuració al benchmark s'especifica com:

```
--player_backend ollama/qwen3:8b
--player_prompt_algo io
--temperature 0.3
```

Arquitectura d'integració. L'agent Pokechamp rep l'estat de la batalla en format text, l'envia al LLM via API HTTP, i interpreta la resposta per generar una ordre de batalla. El flux és:

1. El benchmark llança un *worker subprocess* per cada batch de batalles.
2. L'`AgentFactory` detecta l'identificador (`pokechamp` o `pokellmon`) i importa dinàmicament `get_llm_player` del repositori Pokechamp clonat.
3. L'agent tradueix l'estat de batalla a un *prompt* textual i l'envia a Ollama.
4. El LLM genera un raonament (*Chain of Thought*) i una acció seleccionada.
5. L'acció es tradueix a una ordre de Showdown.

Logging de raonament. A diferència de les heurístiques, els agents LLM generen fitxers de *"Thinking"* que expliquen el raonament darrere de cada decisió. Aquests logs es guarden a `evaluation/results/LLM/` i permeten analitzar la qualitat del raonament estratègic del model (anàlisi de tipo matchups, predicció de moviments rivals, gestió de recursos, etc.).

Aïllament de processos. El principal repte tècnic va ser la gestió de memòria: el fil de fons `POKE_LOOP` de Pokechamp manté referències als objectes de la batalla fins i tot després de `del` + `gc.collect()`, acumulant memòria de forma il·limitada. La solució va ser l'**aïllament per subprocessos**: cada batch de batalles s'executa en un procés Python independent (`worker.py`), i quan el procés acaba, el sistema operatiu reclama *tota* la memòria.

3.4 Fase 1.2b: L'Ascens al Competitiu Real (Dobles VGC)

Tot repte d'IA queda incomplet si no aborda el format oficial de competició: **Dobles 2 vs 2**. La base d'heurístiques dissenyada per Singles resulta invàlida perquè la classe de poke-env per Dobles requereix retornar tuples d'accions coordinades per als dos Pokémon al camp. Dins de `src/p01_heuristics/s02_doubles/` s'ha reestructurat l'arquitectura al patró *Score-then-Combine*:

- **Escala de Candidats (V1 i V2)**: L'algorisme assigna puntuacions de dany teòric a tots els moviments possibles de cada Pokémon i després combina les dues millors accions, verificant que no entrin en conflicte (p. ex. evitant que ambdós Pokémon intentin substituir-se simultàniament).
- **Propagació Espacial (Spread/Focus V3)**: Inclou el multiplicador oficial de dany d'àrea ($\times 0.75$) i la lògica de *Protect* com a eina tàctica defensiva, coordinant quan un aliat es protegeix mentre l'altre ataca.

[Nota: El desenvolupament de les heurístiques de Dobles es troba en fase de finalització i es completarà properes setmanes.]

3.5 Fase 2: Implementació del Pipeline de Reinforcement Learning

Amb les heurístiques com a baselines experts, la segona gran fase del projecte ha consistit a construir un pipeline complet de Reinforcement Learning per entrenar agents neuronals, implementat a `src/p02_rl_models/`.

3.5.1 Vectorització de l'Estat (`StateVectorizer`)

Les xarxes neuronals necessiten dades numèriques com a entrada. El mòdul `vectorizer.py` converteix l'objecte complex `Battle` de poke-env en un **vector float32 de 238 dimensions**, normalitzat a l'interval $[0, 1]$ per estabilitzar l'entrenament. El vector codifica:

Component	Dims	Detall
Pokémon actiu propi	$\sim$33	HP normalitzat, tipus (18-dim multi-hot), status (one-hot), 7 boosts $[-6, 6] \to [0, 1]$.
Pokémon actiu rival	$\sim$33	Mateixa estructura que el propi.
Moviments propis	76	4 slots $\times$ (18 tipus one-hot + 1 potència normalitzada).
Banc propi	6	HP fraccional de cada Pokémon de l'equip.
Banc rival	7	HP dels Pokémon revelats + estimació de vius.
Clima i camp	$\sim$20	Weather i Field com a multi-hot encoding.
Condicions laterals	2×24	Hazards, pantalles, etc. per bàndol (normalitzats).

Table 3.1: Estructura del vector d'observació de 238 dimensions generat pel `StateVectorizer`.

3.5.2 Entorn Gymnasium amb Action Masking

L'entorn `PokemonMaskedEnv` hereta de la classe `SinglesEnv` de poke-env i implementa la interfície estàndard de **Gymnasium**. L'espai d'accions és discret amb 10 accions possibles:

- **Accions 0–3**: Atacs (un per cada slot de moviment).
- **Accions 4–9**: Substitucions (un per cada posició del banc).

Per evitar que l'agent intenti accions il·legals (atacar amb un moviment sense PP, substituir per un Pokémon debilitat), s'utilitza **Action Masking**: cada torn, un vector binari indica quines de les 10 accions són legals. L'algorisme `MaskablePPO` [15] utilitza aquesta màscara per assignar probabilitat zero a les accions invàlides.

3.5.3 Reward Shaping (Disseny de la Recompensa)

La funció de recompensa densa combina múltiples senyals per guiar l'aprenentatge:

- **Base**: Delta de HP (propis i rivals) + bonus per Pokémon derrotat (± 2) + victòria/derrota (± 30).
- **Boosts ofensius**: $+0.3$ per nivell de boost positiu en Atac, Atac Especial o Velocitat.
- **Penalització defensiva**: -0.1 per nivell de debuff propi.
- **Status oponent**: $+0.3$ si el rival actiu té un status debilitant (Burn, Paralysis, Toxic).
- **Hazards**: $+0.5$ per hazard plantat al camp rival; penalització proporcional per hazards propis.
- **Penalització temporal**: -0.02 per torn, per evitar partides infinitament llargues.

3.5.4 Currículum d'Entrenament en 4 Fases

Per evitar que l'agent col·lapsi davant d'oponents massa complexos des de l'inici, s'ha implementat un **currículum progressiu** on la dificultat escala gradualment:

1. **Fase 1 — Fonaments**: Entrenament contra *RandomPlayer*. L'agent aprèn les mecàniques bàsiques: atacar és millor que no fer res, substituir un Pokémon debilitat és obligatori.

2. **Fase 1.5 — Supervivència**: Transició contra *MaxBasePowerPlayer*. L'agent ha d'aprendre a resistir un oponent que sempre ataca amb la potència màxima, desenvolupant nocions defensives.

3. **Fase 2 — Tàctica**: Transferència contra *SimpleHeuristicPlayer*. L'oponent ara avalua tipus i fa substitucions defensives, forçant l'agent a desenvolupar estratègia real.

4. **Fase 3 — Gauntlet**: Entrenament contra **tots els oponents simultàniament** (Random, MaxPower, SimpleHeuristic) amb reward shaping agressiu: victòria ± 100 i penalització temporal de -0.1 per torn, per evitar *stalling* i l'oblit catastròfic (*Cathedral Forgetting*).

En cada fase, es carrega el checkpoint de la fase anterior i es continua l'entrenament amb el nou oponent, permetent una **transferència progressiva** de coneixement.

3.5.5 Arquitectura del Model i Infraestructura GPU

L'agent utilitza **MaskablePPO** de la llibreria `sb3-contrib` [15] amb els següents hiperparàmetres:

- **Xarxa**: MLP amb 2 capes ocultes de 256 neurones cadascuna (`net_arch=[256, 256]`).
- **Learning rate**: 3×10^{-4}.
- **Steps per batch**: 2048.
- **Entropy coefficient**: 0.01 (incentiva l'exploració en les primeres fases).
- **Discount factor (γ)**: 0.99.

L'entrenament s'executa en una màquina amb una **GPU RTX 2080** (8 GB VRAM) i 16 nuclis de CPU. Per maximitzar el throughput, es despleguen **10 instàncies paral·leles** de Showdown (ports 8000–8009) amb `SubprocVecEnv`, permetent recopilar experiència 10 vegades més ràpid que amb un sol servidor.

3.6 Treball Futur: Aprenentatge Basat en Datasets de Partides

Més enllà del Reinforcement Learning pur, una línia de treball futur prometedora consisteix a explotar **datasets de partides existents** per entrenar agents mitjançant tècniques d'*Imitation Learning* o *Behavioral Cloning*.

El repositori Pokechamp [12] inclou internament un conjunt de replays de partides competitives que registren seqüències completes d'estats i accions. La idea és:

1. **Extracció**: Parsejar els replays per obtenir parelles (s_t, a_t) d'estat-acció de jugadors experts.

2. **Supervisió**: Entrenar una xarxa neuronal per predir l'acció experta a_t donat l'estat vectoritzat $\mathbf{o}_t$ (usant el `StateVectorizer` ja implementat i la loss de cross-entropy).

3. **Fine-tuning amb RL**: Utilitzar el model pre-entrenat com a punt de partida per a les fases de RL, combinant el coneixement humà amb l'aprenentatge autònom.

Aquesta aproximació podria accelerar significativament la convergència del RL: en lloc de descobrir des de zero que "un atac de tipus Aigua és efectiu contra un Pokémon de Foc", l'agent partiria d'un *prior* expert, i el RL només hauria de refinar les decisions en situacions ambigües o no representades al dataset.

Chapter 4

Resultats i Avaluació de les Heurístiques

Aquest capítol presenta els resultats obtinguts de l'avaluació exhaustiva d'ambdós entorns de simulació (Singles i Doubles). S'analitza l'efectivitat i fiabilitat de les versions estructurades respecte als baselines, utilitzant les eines de *benchmark* internes de l'entorn Core, desenvolupades sota el disseny *Round-Robin* automatitzat explicat prèviament.

4.1 Infraestructura d'Execució: Gestió de Recursos

Un aspecte crític del procés de benchmarking massiu és la **gestió de memòria RAM**. Cada batalla Pokémon consumeix memòria tant al procés Python (objectes Battle, Players) com al servidor Showdown (processos `room-battle.js` de Node.js). Sense mecanismes de neteja, la memòria creix de forma il·limitada.

La Figura 4.1 mostra el perfil de consum de RAM durant l'execució d'un benchmark complet de Singles:

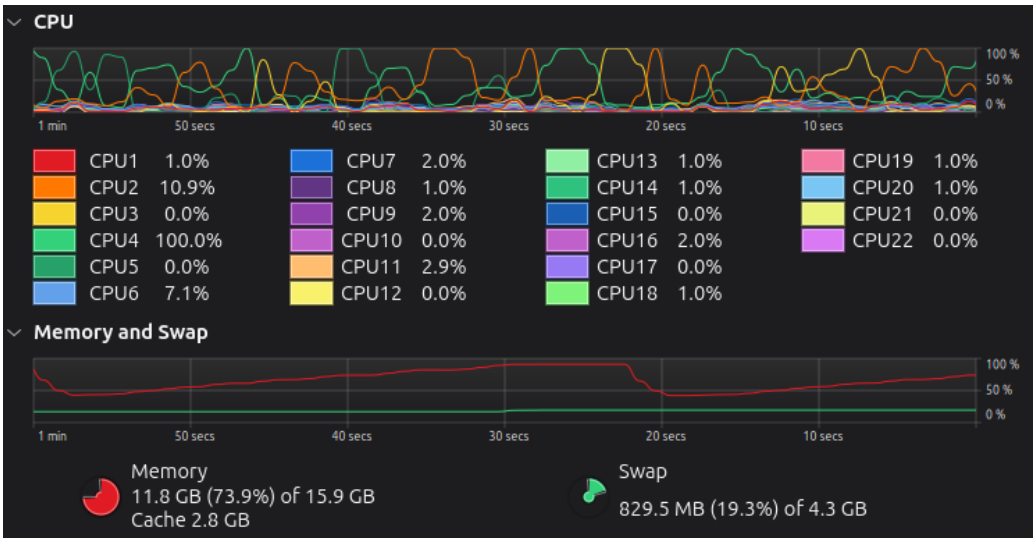

Figure 4.1: Consum de RAM durant un benchmark de Singles. S'observa com la memòria puja progressivament durant cada matchup (acumulació de batalles) i cau bruscament quan el servidor Showdown es reinicia i el Garbage Collector de Python s'executa entre grups de matchups.

El patró de dents de serra reflecteix el mecanisme de `--restart-every`: cada N matchups completats, l'orquestrador mata i reenlaira els servidors Showdown, alliberant tots els processos

`room-battle.js` acumulats. Combinat amb l'aïllament per subprocessos (cada batch de batalles en un procés Python independent), aquest sistema permet executar desenes de milers de batalles sense excedir els límits de memòria de la màquina.

4.2 Avaluació Exhaustiva al Format 1 vs 1 (Singles)

L'anàlisi sistemàtic del format *Singles* va procedir a confrontar iterativament tots els agents heurístics dissenyats (des de *v1* fins a *v6*) i els *baselines* absoluts de l'entorn.

Les proves es van estructurar en sèries massives, llançant matrius d'enfrontaments per cada possible par (ex. $v6$ vs *Random*, $v3$ vs *MaxBasePower*), on els milers d'execucions es van processar i sintetitzar de forma paral·lela obtenint el guanyador o perdedor teòric. En l'observació del percentatge de victòries net (*Win Rate*), la tendència generalitzadora marca clarament una **progressió evolutiva de disseny encertada**. Generacions inicials com la $v1$ no obtenien victòries aclaparadores més enllà del 55%-60% en intentar basar decisions únicament en regles ofendents de dany simple respecte atacs a l'atzar del *RandomPlayer*. La introducció en modelat teòric profund amb *The Tracker* ($v3$) i l'establiment expert d'incorporació de tipologies defensives del motor al *The Champion* ($v6$) augmentava de cop en un rendiment tàcit els índexs per sobre del 90%-95% quant es combatien davant els elements de *poke-env*.

El pes de la Estocasticitat: Malgrat crear intel·ligències heurístiques altament sofisticades, de la visió general numèrica se n'extreu que una **victòria del 100% és intrínsecament impossible en aquest entorn i tipus d'agents reaccionaris purs**. Això resideix essencialment a la incommensurable potència de la inobservabilitat, afegit al RNG absolut del joc i al dany latent d'encerts crítics aleatoris que es poden atorgar de manera favorable totalment a un element de sort inferior que utilitzi combinatòries subòptimes (*baselines*). L'anàlisi de matrius presentades indica clarament aquesta resistència asimptòtica.

benchmarks_singles_v3/placeholder_singles_heatmap.png

Figure 4.2: Matriu de calor mostrant el percentatge de victòries (*Win Rate*) de tots els encreuaments de *Singles* evaluant baselines vs. $v1$-$v6$. Es pot lligar com els iteratius majors assoleixen superioritat al marge inferior esquerre.

4.3 Avaluació al Format 2 vs 2 (Doubles)

De manera similar, es va testejar la fiabilitat al *Core Doubles* aprofitant el disseny *Score-then-Combine*. Sota la multiplicitat d'arbre de l'enginyeria, el càlcul d'eficiència es restringia exclusivament en els prototips primigenis del llinatge *v1* al *v3* per simplificació experimental i focalització d'estudi previ al model DRL per xarxes neurals.

La visualització empírica als tornejos demostrà de nou que hi hagué una fita creixent en els *baselines* quan es presentaven múltiples entitats objectives (*multi-target*). Exigir decisions conjuntes penalitza radicalment als *RandomPlayers* on sovint aquests malgastaven els atacs atacant-se simultàniament les pròpies files si es produïen errors del llançament a l'atzar respecte el tipus objectiu (foc amic). No obstant això, el rendiment estabilitzador a través de combinant parelles òptimes per pes amb el *v3 Doubles* garanteix ser base per establir estratègies pre-predictores suficients i emmagatzemar la pre-avaluació requerida que encoratjarà bases de model entrenables i més complexes.

benchmarks_doubles_v2/placeholder_doubles_heatmap.png

Figure 4.3: Avaluació exhaustiva al format *Doubles*. La progressió de $v1$ fins a $v3$ en el comportament de *Score-then-Combine* contra els *baselines* teòrics de *poke-env*.

Bibliography

[1] Christopher Berner et al. "Dota 2 with Large Scale Deep Reinforcement Learning". In: *arXiv preprint arXiv:1912.06680* (2019). DOI: https://doi.org/10.48550/arXiv.1912.06680.

[2] Greg Brockman et al. "OpenAI Gym". In: *arXiv preprint arXiv:1606.01540* (2016).

[3] Noam Brown and Tuomas Sandholm. "Superhuman AI for heads-up no-limit poker: Libratus beats top professionals". In: *Science* 359.6374 (2018), pp. 418–424. DOI: https://doi.org/10.1126/science.aao1733.

[4] Noam Brown and Tuomas Sandholm. "Superhuman AI for multiplayer poker". In: *Science* 365.6456 (2019), pp. 885–890. DOI: https://doi.org/10.1126/science.aay2400.

[5] Cameron B. Browne et al. "A Survey of Monte Carlo Tree Search Methods". In: vol. 4. 1. 2012, pp. 1–43. DOI: https://doi.org/10.1109/TCIAIG.2012.2186810.

[6] Bulbapedia Contributors. *Pokémon (species) — Bulbapedia*. Enciclopèdia completa de la franquícia Pokémon. 2024. URL: https://bulbapedia.bulbagarden.net/wiki/Pok%5C%C3%5C%A9mon_(species).

[7] Bulbapedia Contributors. *Statistic — Bulbapedia, The community-driven Pokémon encyclopedia*. [Online; accessed 4-March-2026]. 2024. URL: https://bulbapedia.bulbagarden.net/wiki/Stat.

[8] Murray Campbell, A.Joseph Hoane, and Feng-hsiung Hsu. "Deep Blue". In: *Artificial Intelligence* 134.1 (2002), pp. 57–83. ISSN: 0004-3702. DOI: https://doi.org/10.1016/S0004-3702(01)00129-1. URL: https://www.sciencedirect.com/science/article/pii/S0004370201001291.

[9] Wolfe Glick. *WolfeyVGC — Pokémon VGC Educational Content*. https://www.youtube.com/@WolfeyVGC. Doble campió del món de VGC (2016, 2019). 2024.

[10] Donald E. Knuth and Ronald W. Moore. "An analysis of alpha-beta pruning". In: vol. 6. 4. Elsevier, 1975, pp. 293–326. DOI: https://doi.org/10.1016/0004-3702(75)90019-3.

[11] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *Nature* 521.7553 (2015), pp. 436–444. DOI: https://doi.org/10.1038/nature14539.

[12] Champion Lee et al. *Pokechamp: A Collection of Pokémon Battle AI Agents*. https://github.com/champion-lee/pokechamp. Inclou agents basats en MCTS, Minimax, Xarxes Neuronals i heurístiques avançades. 2024.

[13] Guangcong Luo and Smogon Community. *Pokémon Showdown!* https://pokemonshowdown.com/. 2011.

[14] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: https://doi.org/10.1038/nature14236.

[15] Antonin Raffin et al. *Stable-Baselines3 Contrib: Reliable Reinforcement Learning Implementations*. https://github.com/Stable-Baselines-Team/stable-baselines3-contrib. Inclou MaskablePPO per a entorns amb action masking. 2021.

[16] Haris Sahovic. *poke-env: A Python Interface for Creating Pokémon Showdown Bots*. https://github.com/hsahovic/poke-env. 2020. URL: https://github.com/hsahovic/poke-env.

[17] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017). DOI: https://doi.org/10.48550/arXiv.1707.06347.

[18] David Silver et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play". In: *Science* 362.6419 (2018), pp. 1140–1144. DOI: https://doi.org/10.1126/science.aar6404.

[19] David Silver et al. "Mastering the game of Go with deep neural networks and tree search". In: *Nature* 529.7587 (2016), pp. 484–489. DOI: https://doi.org/10.1038/nature16961.

[20] Richard D. Smallwood and Edward J. Sondik. "The optimal control of partially observable Markov processes over a finite horizon". In: *Operations Research* 21.5 (1973), pp. 1071–1088. DOI: https://doi.org/10.1287/opre.21.5.1071.

[21] Smogon University. *Damage Formula*. [Online; accessed 4-March-2026]. 2024. URL: https://www.smogon.com/dp/articles/damage_formula.

[22] Smogon University. *Smogon Strategy Pokédex*. https://www.smogon.com/. Comunitat principal d'estratègia Pokémon competitiva. 2024.

[23] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book.html.

[24] The Pokémon Company International. *History of the Video Game Championships*. https://www.pokemon.com/us/play-pokemon/about/vgc/. 2024.

[25] The Pokémon Company International. *Pokémon World Championships Prize Pool and Awards*. https://www.pokemon.com/us/play-pokemon/worlds/2024/vgc-competitor-info/. 2024.

[26] Oriol Vinyals et al. "Grandmaster level in StarCraft II using multi-agent reinforcement learning". In: *Nature* 575.7782 (2019), pp. 350–354. DOI: https://doi.org/10.1038/s41586-019-1724-z.

[27] John Von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 2007. URL: https://press.princeton.edu/books/paperback/9780691130613/theory-of-games-and-economic-behavior.